เรื่อง Sorting Master

จัดทำโดย

นายธนารัตน์ คุ้มผล 67332310304-8 ECP3R

นายธนกฤต อินทร์ผึ่ง 67332310048-3 ECP3R

เสนอ

อาจารย์ ประภาส ผ่องสนาม

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา

การออกแบบระบบอินเทอร์เน็ตของสรรพสิ่ง

ภาคเรียนที่ 2 ปีการศึกษา 2568

มหาวิทยาลัยเทคโนโลยีราชมงคลอีสานวิทยาเขตขอนแก่น

อำเภอเมือง จังหวัดขอนแก่น

รายงานแนะนำเกมการศึกษา: Sorting Master (ยอดนักคัดแยกขยะ)

1. ที่มาและความสำคัญ

ในปัจจุบัน ปัญหาขยะล้นเมืองและมลภาวะทางสิ่งแวดล้อมเป็นปัญหาที่ทวีความรุนแรงมากขึ้น สาเหตุหนึ่งมาจากการทิ้งขยะไม่ถูกที่และขาดการคัดแยกขยะอย่างถูกวิธีตั้งแต่ต้นทาง การสร้างความตระหนักรู้และปลูกฝังวินัยในการคัดแยกขยะจึงเป็นสิ่งสำคัญ

ด้วยเหตุนี้ คณะผู้พัฒนาจึงได้สร้างสรรค์เกม "Sorting Master" ซึ่งเป็นเกมจำลองสถานการณ์การคัดแยกขยะในโรงงาน โดยประยุกต์ใช้ความรู้ด้านการเขียนโปรแกรม (Python & Pygame) มาบูรณาการเข้ากับเนื้อหาด้านสิ่งแวดล้อม เพื่อให้ผู้เล่นได้เรียนรู้วิธีการคัดแยกขยะทั้ง 4 ประเภทผ่านการเล่นเกมที่สนุกสนาน ท้าทาย และเข้าใจง่าย

2. วัตถุประสงค์

1. เพื่อให้ผู้เล่นมีความรู้ความเข้าใจเกี่ยวกับประเภทของขยะ และสามารถคัดแยกขยะลงถังได้อย่างถูกต้อง
2. เพื่อฝึกฝนทักษะความไวของการทำงานประสานกันระหว่างสายตาและมือ (Hand-Eye Coordination)
3. เพื่อประยุกต์ใช้ภาษา Python และไลบรารี Pygame ในการพัฒนาสื่ออินเทอร์แอคทีฟ
4. เพื่อสร้างความสนุกสนานและส่งเสริมการใช้เวลาว่างให้เกิดประโยชน์

3. อุปกรณ์และความต้องการของระบบ

เปรียบเสมือนกระดานและเบี้ยในเกม A-Math เกม Sorting Master ต้องการอุปกรณ์ในการประมวลผลและแสดงผลดังนี้:

- ฮาร์ดแวร์: เครื่องคอมพิวเตอร์, เมาส์ (สำหรับการลากและปล่อยขยะ), และหน้าจอแสดงผล
- ซอฟต์แวร์: ระบบปฏิบัติการ Windows/macOS/Linux ที่ติดตั้งโปรแกรม Python (เวอร์ชัน 3.x ขึ้นไป) และไลบรารี Pygame
- ไฟล์ระบบเกม: ไฟล์ชุดคำสั่ง main.py และชุดภาพกราฟิก (Assets) เช่น พื้นหลังโรงงาน สายพาน ถังขยะ 4 สี และรูปภาพขยะประเภทต่างๆ

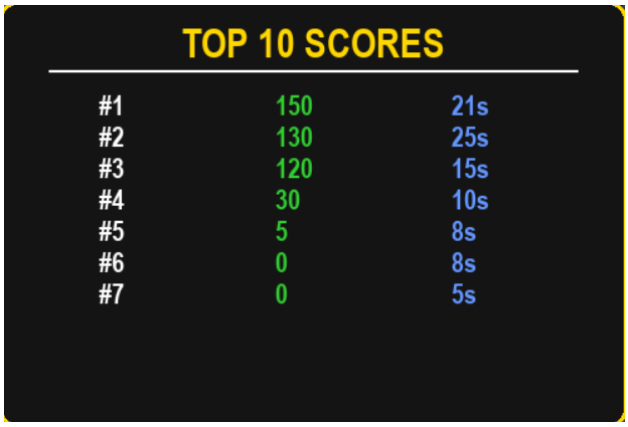
4. องค์ประกอบภายในเกม

1. สายพานลำเลียง (Conveyor Belt): จุดที่ขยะจะถูกสุ่มปล่อยลงมาอย่างต่อเนื่อง

2. ขยะ 4 ประเภท: * ขยะรีไซเคิล (Recycle)
   - ขยะเปียก (Wet)
   - ขยะทั่วไป (General)
   - ขยะอันตราย (Hazardous)

3. ถังขยะ 4 สี: รองรับขยะตามประเภทที่กำหนดไว้

4. กระดานคะแนน (Leaderboard): ระบบบันทึกสถิติผู้เล่นที่ทำคะแนนสูงสุด 10 อันดับแรก (Top 10) พร้อมบันทึกเวลาที่ใช้เล่น



5. กติกาและวิธีการเล่น

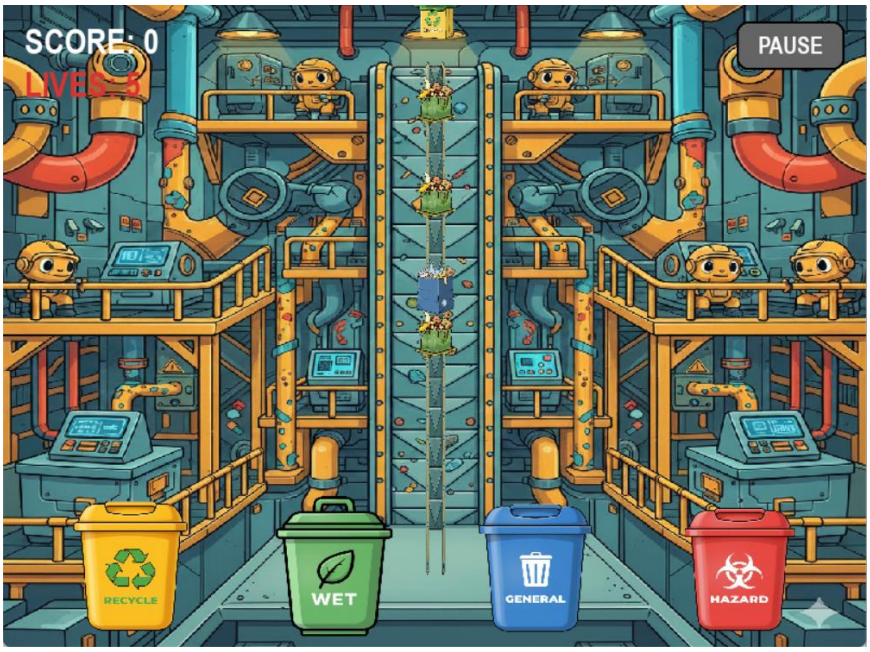
การเริ่มต้นเกม:

1. เข้าสู่หน้าเมนูหลัก (Main Menu) ผู้เล่นจะพบชื่อเกมและกระดานแสดงคะแนนสูงสุด (หากยังไม่มีประวัติการเล่น ระบบจะแสดงข้อความเชิญชวน)

2. คลิกที่ปุ่ม "START GAME" เพื่อเริ่มเกม

วิธีการเล่น (Gameplay):

1. เมื่อเกมเริ่มขึ้น ขยะประเภทต่างๆ จะสุ่มตกลงมาตามสายพานลำเลียงตรงกลางหน้าจอ
2. ให้ผู้เล่นใช้เมาส์ คลิกซ้ายค้างไว้ (Click & Hold) ที่ชิ้นขยะ เพื่อทำการหยิบขยะชิ้นนั้นขึ้นมา
3. ลาก (Drag) ขยะไปยังถังขยะที่มีสีและประเภทตรงกับขยะชิ้นนั้น
4. ปล่อยเมาส์ (Drop) เพื่อทิ้งขยะลงถัง



การคิดคะแนนและการสิ้นสุดเกม:

- ทิ้งถูกต้อง: หากผู้เล่นลากขยะลงถังได้ตรงประเภท จะได้รับคะแนนสะสม +10 คะแนน ต่อชิ้น
- ทิ้งผิดพลาด: หากผู้เล่นลากขยะใส่ผิดถัง หรือ ปล่อยให้ขยะไหลตกขอบหน้าจอด้านล่างไปโดยไม่ได้จัดเก็บ จะสูญเสีย พลังชีวิต (Lives) 1 หน่วย (โดยคะแนนจะไม่ถูกหัก)
- การหยุดพัก: ผู้เล่นสามารถกดปุ่ม "PAUSE" เพื่อหยุดเกมชั่วคราวได้ โดยระบบจะหยุดเวลารวมของเกมไว้
- การจบเกม: เกมจะสิ้นสุดลงทันที (Game Over) เมื่อพลังชีวิต (Lives) ลดลงเหลือ 0 ระบบจะทำการสรุปคะแนนและเวลาที่ทำได้ พร้อมบันทึกลงกระดานผู้นำโดยอัตโนมัติ

6. ประโยชน์ที่ได้รับ

1. ด้านวิชาการ: ผู้พัฒนาได้ฝึกฝนทักษะการเขียนโปรแกรมเชิงวัตถุ (OOP), การจัดการสถานะของเกม (State Machine), และการออกแบบ UI/UX
2. ด้านผู้เล่น: ผู้เล่นสามารถจดจำการแยกประเภทขยะได้แม่นยำขึ้นผ่านระบบการให้รางวัลในเกม (Gamification) ซึ่งสามารถนำไปประยุกต์ใช้กับการทิ้งขยะในชีวิตจริงได้
3. ด้านอารมณ์และสังคม: ช่วยลดความเครียด ฝึกสมาธิ และสามารถใช้เป็นกิจกรรมแข่งขันทำคะแนนสูงสุด (High Score) ภายในครอบครัวหรือกลุ่มเพื่อนได้

7. โครงสร้างข้อมูลและอัลกอริทึมที่ใช้ในการพัฒนา (Data Structure & Algorithm)

ในการพัฒนาเกม Sorting Master มีการเลือกใช้โครงสร้างข้อมูลและอัลกอริทึมที่เหมาะสม เพื่อให้เกมทำงานได้อย่างมีประสิทธิภาพและจัดการหน่วยความจำได้ดี ดังนี้

7.1 โครงสร้างข้อมูล (Data Structures)

1. ลิสต์แบบไดนามิก (Dynamic Array / List):
    - items = []: ใช้เก็บวัตถุ (Object) ของขยะที่กำลังอยู่บนหน้าจอ ลิสต์มีความยืดหยุ่นสูง สามารถเพิ่มขยะชิ้นใหม่ (Append) เมื่อสายพานปล่อยขยะ และลบออก (Remove) ได้ทันทีเมื่อผู้เล่นทิ้งขยะลงถังหรือขยะตกขอบจอ
    - bins และ scores: ใช้เก็บข้อมูลของถังขยะและประวัติคะแนนสูงสุด 10 อันดับ เพื่อการวนลูปแสดงผลบนหน้าจอ
2. ดิกชันนารี (Dictionary / Hash Map):
    - item_images และ bin_images: ใช้โครงสร้างแบบ Key-Value Pair ในการจับคู่ "ชื่อประเภทขยะ" (เช่น "WET", "RECYCLE") เข้ากับ "ไฟล์รูปภาพ" ทำให้ระบบสามารถดึงรูปภาพมาแสดงผลได้อย่างรวดเร็วในระดับ O(1) โดยไม่ต้องใช้เงื่อนไข if-else ซ้อนกันหลายชั้น
3. การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Structure):
    - Class Garbage: ใช้โครงสร้างคลาสเพื่อสร้างแม่พิมพ์ของขยะ โดยห่อหุ้ม (Encapsulation) ข้อมูลต่างๆ ไว้ด้วยกัน เช่น ชนิดของขยะ, ตำแหน่ง (Rect), ความเร็ว (Speed), สถานะการถูกจับลาก (Dragging) และฟังก์ชันการเคลื่อนที่ลงล่าง (move())

7.2 อัลกอริทึม (Algorithms)

1.อัลกอริทึมการตรวจจับการชน (Collision Detection Algorithm):

- เกมใช้อัลกอริทึม AABB (Axis-Aligned Bounding Box) ผ่านฟังก์ชันของ Pygame
- collidepoint(x, y): ใช้เช็คว่า "พิกัดของเมาส์" อยู่ในพื้นที่กรอบสี่เหลี่ยมของชิ้นขยะหรือไม่ (ใช้ตอนคลิกหยิบขยะ)
- colliderect(rect): ใช้เช็คว่า "กรอบสี่เหลี่ยมของขยะ" ซ้อนทับกับ "กรอบสี่เหลี่ยมของถังขยะ" หรือไม่ (ใช้ตอนปล่อยขยะลงถัง)

2.อัลกอริทึมการเรียงลำดับ (Sorting Algorithm):

- ในระบบกระดานผู้นำ (Leaderboard) ใช้ฟังก์ชันการเรียงลำดับ scores.sort(key=lambda x: x["score"], reverse=True) ซึ่งเบื้องหลังของ Python คือ อัลกอริทึม Timsort (การผสมผสานระหว่าง Merge Sort และ Insertion Sort) เพื่อเรียงคะแนนผู้เล่นจากมากไปน้อยอย่างรวดเร็ว

3.อัลกอริทึมเครื่องจักรสถานะจำกัด (Finite State Machine - FSM):

- ใช้ควบคุมลอจิกการไหลของเกม (Game Flow) ผ่านตัวแปร state โดยแบ่งเป็น 4 สถานะหลักคือ 'MENU', 'PLAYING', 'PAUSED', และ 'GAMEOVER' ช่วยให้ระบบรู้ว่าในแต่ละเฟรม (Frame) ควรวาดภาพและรับค่าคำสั่งอะไรบ้าง

4.การวนลูปย้อนกลับ (Reverse Iteration):

- ในระบบ Drag & Drop ใช้คำสั่ง for item in reversed(items): ในการค้นหาว่าผู้เล่นคลิกโดนขยะชิ้นไหน การวนลูปจากหลังมาหน้าทำให้อัลกอริทึมตรวจจับขยะที่อยู่ "บนสุด" (ชิ้นที่ถูกวาดทีหลังสุดและทับชิ้นอื่นอยู่) ได้อย่างถูกต้องสมจริง

5.อัลกอริทึมการสุ่มและน่าจะเป็น (Randomization & Probability):

- ใช้ random.random() < 0.02 เพื่อสร้างโอกาส 2% ในแต่ละเฟรมที่จะมีขยะชิ้นใหม่ตกลงมา ทำให้ระยะห่างของขยะแต่ละชิ้นดูเป็นธรรมชาติ ไม่ตายตัว

- ใช้ random.uniform() ในการสุ่มความเร็วของขยะแต่ละชิ้นให้ตกลงมาไม่เท่ากัน เพิ่มความท้าทายให้ผู้เล่น